Memoria Práctica 2 WepSim

Grado de Ingeniería Informática

Segundo año, grupo 85

Jaime Sánchez Sánchez 100522207

Ana Díaz Jiménez 100522155

Índice

Contenido

Ejercicio 1 3

Ejercicio 2 6

Problemas encontrados 8

Conclusiones 9

Ejercicio 1

Nombre de la instrucción	Diseño de instrucciones en lenguaje RT	Señales de control a activar en cada ciclo de cada instrucción	Decisiones de diseño
lx RR1, U32	1. MAR <- PC 2. MBR <- MP[MAR], PC <- PC +4 3. RR1 <- MBR	1. T2, C0 2. TA, R, BW=11, M1=1, C1=1, M2, C2 3. T1, LC=1, MR=0, SELC=10101, A0=1, B=1, C=0	Para llevar a cabo esta instrucción lo primero es cargar la segunda palabra (pues es una instrucción de 64 bits) sin alterar el contenido de IR, por tanto decidimos hacer un "fetch" pero sin meter el valor de MBR en IR y directamente cargándolo en el registro de la instrucción.
call U32	1. PC <- MAR 2. MBR <- MP[MAR], PC <- PC +4 3. RT1 <- MBR 4. SP <- SP – 4, MAR <- SP 5. MP[SP]<- PC 6. PC <- RT1	1. T2, C0 2. TA, R, BW=11, M1=1, C1=1, M2, C2 3. T1, C4 4. MR=1, SelA=00010, MA=0, MB=10, SelCop=1011, MC, T6, SelC=00010, LC=1, C0, SELP=11, M7, C7 5. T2, C1 6. Ta, BW=11, Td, W,T4, M2=0, C2, A0=1, B=1, C=0	Al ser una instrucción de 64 bits también, optamos por hacer de nuevo un "fetch" sin llevar la segunda palabra a IR y tener el valor a cargar en MBR directamente, guardándolo en RT1. Luego le restamos a la pila 4 para poder cargar el valor de PC ahí y así recuperarlo más tarde. Finalmente metemos el valor de la posición de memoria que estaba en RT1y lo insertamos en PC, pasando a ejecutar la subrutina ahí almacenada.
return	1. MAR <- SP 2. MBR <- MP [SP], SP <- SP + 4 3. PC <- MBR	1. SelA= 00010, MR, T9, C0 2. Ta, R, BW=11, M1, C1, SelA=00010, MA=0,MB=10, MR, SelCop=1010, MC=1, T6, SelC=00010, LC	Como con esta instrucción queremos volver a la rutina en la que nos encontrábamos con el valor almacenado en la pila, desapilamos (sumamos 4 a la pila) para devolverla

		3. T1, M2=0, C2, A0=1, B=1, C=0	a su antiguo estado, y metemos la dirección de la pila en MAR para sacar el valor contenido en ella, luego ese valor lo cargamos en PC para que lea la instrucción de la rutina siguiente.
stop	1. PC <- zero, SP <- zero	1. SelA=00000, T9, SelC= 00010, MR, LC, M2=0, C2	Como queremos cargar el valor 0 en SP y PC, parando el programa, sacamos el valor del registro zero (que es cero) y lo metemos tanto en PC como en la pila.
vfill RR1, (RR2), U8	1. RT2 <- SR 2. RT1 <- zero 3. MBR <- U8, RT3 <- RT2 4. RT2 <- RR2, MAR <- RR2 5. inicio: MP[MAR]<- MBR 6. MAR <- RR2 + RT1, if RR1 - RT1 == 0 -> fin 7. RT1 <- RT1 + 1, return inicio 8. fin: RR2 <- RR2 + RR1 9. RR1 <- 0 10. SR <- RT3	1. T8, C5 2. SelA=00000, MR=1, T9, C4 3. SE=1, OFFSET=0, SIZE=01000,T3, M1=0, C1, MB=01, MA=1, SelCop=1010, MC, C6 4. MR=0,SelB=10000, T10, C5, C0 5. Td, Ta, BW=00, W, MA=1, SelB=10101, MB=00, SelCop=1011, MC, SelP=11, M7=1,C7 6. A0=0, B=0, C=110, MADDR=fin, MB=01, MA=1, SelCop=1010, MC, T6, C0 7. MA=1, MB=11, SelCop=1010, MC, T6, C4, A0=0, B=1, C=0, MADDR=inicio 8. SelA=10101, SelB=10000, MA=0, MB=00, SelCop=1010,	Esta instrucción almacena en una posición de memoria, que consiste del contenido RR2 sumándole 1 tantas veces como el valor de RR1, el valor U8. Por tanto lo primero que hacemos es almacenar SR en RT2 y vaciar RT1, para que crezca de 0 hasta el valor que tenga RR1 a base de sumarle 1 cada ciclo del bucle. Luego almacenamos el valor U8 en MBR, pues va a ser el valor que va a ir almacenado en todas las direcciones y movemos SR de RT2 a RT3. Metemos RR2 en MAR y en RT2, para almacenar la primera vez el valor U8 en el addr RR2 (porque i vale 0). Iniciamos en bucle cargando en la dirección de

		MR=0, MC, T6, SelC=10000, LC 9. ExCode=0000, T11, SelC=10101, LC 10. T7, M7=0, C7, A0=1, B=1, C=0	memoria el valor U8 en MBR Ahora sumamos el registro RT2 (que contiene el contenido de RR2)con el contenido de RT1 y lo metemos en mar, para cargarle luego el valor de U8. Le restamos a RR1 el valor de RT1, para que, en caso de que sea cero el resultado, salimos del bucle, en el otro caso actualizamos RT1 sumándole 1 si la resta no resulta 0 y saltamos a la etiqueta inicio. Si se da el caso de que la resta se vuelve 0, con Excode metemos el valor 0 en RR1 como se nos pide, antes habiendo claro sumado el contenido de RR1 y RR2 y almacenado este en RR2.
vadd RR1, (RR2), RR3	1. RT2 <- SR 2. RT1 =0, SR=SR+0 3. inicio: Reg2 + RT1 -> MAR 4. addr[MAR]->MBR, reg1-i->SR 5. SR==0->fin, MBR->RT2 6. RT2+reg3-> MBR 7. RT1=1+RT1, MBR-> addr[MAR], y salto a incio	1. T8, C5 2. SelA=00000, MR=1, T9, C4 3. SE=1, OFFSET=0, SIZE=01000,T3, M1=0, C1, MB=01, MA=1, SelCop=1010, MC, C6 4. Ta, R, BW=00, M1, C1, SelB=10101, MR=0, MA=1, MB=00, SelCop=1011, MC, SelP=11, M7=1, C7	En esta instrucción almacena desde la dirección de R2 el número en el registro R3 las R1 veces. Primero guardamos SR en RT2, para así no perder el valor, después limpiamos RT1 y sumamos “0” a RT2 para mover SR a RT3, en el siguiente paso empezamos el bucle sumando reg2 e “i” y guardandolo en MAR, después buscamos el valor en la memoria y lo devolvemos en MBR mientras restamos RT1 e i para ver si es

	8. fin: Rr2 ←BR[RR2] + BR[RR1] 9. R1=0 10. SR=RT3	5. A0=0, B=0, C=110, MADDR=fin2, T1, C5 6. SelA=01011, MR=0, MA=0, MB=01, SelCop=1010, MC, T6, M1=0, C1 7. Ta, W, Td, BW=00, MA=1, MB=11, SelCop=1010, MC, T6, C4, A0=0, B=1, C=0, MADDR=inicio2 8. SelA=10101, SelB=10000, MA=0, MB=00, SelCop=1010, MR=0, MC, T6, SelC=10000, LC 9. ExCode=0000, T11, SelC=10101, LC 10. T7, M7=0, C7, A0=1, B=1, C=0	0, en la siguiente instrucción, si ha sido 0, acabamos el bucle y guardamos el valor de MBR que hemos obtenido en RT2, por sí seguimos avanzando, después sumamos el valor que está en RT2, con reg3 y lo guardamos en MBR para en el siguiente paso guardar ese valor en la memoria y además actualizar RT1 +1, y hacer un salto al inicio del bucle.

Ejercicio 2

Diferencias

La diferencia principal entre ambos códigos es que el primero está codificado con instrucciones de ensamblador cuyo microcódigo se nos otorga, mientras que el segundo posee instrucciones en microcódigo que debemos codificar nosotros mismos.

Dicho esto, la principal diferencia radica en que crear el microcódigo es mucho más complejo que una simple instrucción de ensamblador, pero sin embargo esta complejidad nos permite moldear el comportamiento del procesador hacia lo que más nos pueda beneficiar para nuestro programa, como es el caso de la práctica, que a pesar de poderse solventar con instrucciones de ensamblador, al tener instrucciones más vitaminadas podemos dejar el código de ensamblador más limpio además de que este último es mucho más optimizado compaginandolo con las instrucciones de microcódigo,

Ventajas e inconvenientes

Las ventajas que podemos encontrar del código uno respecto del dos es que no requerimos de instrucciones extra y se puede crear el código con las instrucciones básicas que se nos dan, mientras que para el segundo código debemos codificar las instrucciones necesarias para su funcionamiento en microcódigo nosotros mismos, dando más margen a error y a malfuncionamiento del programa.

Sin embargo, el segundo código es más eficiente que el primero al tener instrucciones específicas para su ejecución, haciendo que se requiera menos ciclos de reloj para su ejecución y no abusando de la repetición de instrucciones en assembly.

Propuestas de mejora

Con esta práctica, una de nuestras principales ambiciones ha sido exprimir al máximo el simulador para ver hasta dónde podemos llegar con el microcódigo, el cual hemos tratado de optimizar al máximo, para ello hemos llegado a hacer instrucciones desde 0 con el fin de bajar el número de ciclos de reloj, haciendo varias optimizaciones:

```
segundo: 54959, 8423
tercero: 54959, 7919
cuarto: 54959, 7415
quinto: 54959, 6914
sexta: 54959, 6404
final: 54959, 6410
final_final: 54959, 6410 (con SR)
```

logrando reducir los ciclos de reloj hasta 6410.
Estamos seguros de que se puede llegar a optimizar más el microcódigo, pero nos hemos detenido aquí porque creemos que tenemos unos resultados más que aceptables con una mejora del 857,39% .

Tabla de ciclos de reloj

	Ciclos de reloj sin extensión (valor en s1)	Ciclos de reloj sin extensión (valor en s2)	Mejora (%)
Semáforo 8 x 24	54959	6410	857,39%

¿merece la pena disponer de la extensión del ejercicio 1?

En vista del porcentaje de mejora del código implementado con las instrucciones del ejercicio uno respecto al código sin ellas, la ejecución del segundo código es notoriamente más eficiente que el primer código, superándolo en 48549 ciclos de reloj. Es por esto que merece bastante la pena el uso de las instrucciones del ejercicio uno a la hora de llevar a cabo un programa más rápido, más eficiente y menos complejo.

Problemas encontrados

1. Uno de los principales problemas que hemos encontrado ha sido la nula funcionalidad de nuestro return, el cual no guardaba bien los datos en SP, haciendo que no se pudiesen hacer más de 2 returns anidados, teniendo que hacer pruebas hasta encontrar ese error.
2. Al principio del ejercicio, nada más probar el vadd y el vfill, no nos rellenaba el dato en la matriz de cuadrados, esto era porque habíamos puesto un acceso a MAR de más y no llegaba a almacenar bien el valor en la matriz
3. Nos dimos cuenta luego que vfill y vadd seguían sin funcionar correctamente, a pesar de estar lógicamente correcto desde nuestro punto de vista, perdiendo horas en buscar el error, cuando el error en ambas era principalmente que BW era igual a “01”, porque pensábamos que era el valor más pequeño, en lugar de ser “00”.
4. El call no llegaba a direccionar bien el PC desde SP.

Conclusiones

El trabajo nos ha mostrado lo preciso que hay que ser a la hora de escribir instrucciones en microcódigo, pues un error o acceso equivocado puede alterar completamente el resultado. Muchas veces el error residía en eso o en la mala ordenación de las instrucciones, que provocaba un fallo catastrófico.

La práctica aun así ha resultado satisfactoria y nos ha ayudado a entender el funcionamiento del microcódigo y a perfeccionar su uso con ella.

Resulta interesante saber cómo funcionan los saltos dentro de los registros de un ordenador y los diferentes elementos que se ejecutan al realizar un proceso tan común como cambiar de color una matriz según si se utilizan instrucciones concretas diseñadas especialmente para su ejecución o si se emplean instrucciones generales no específicas. Era notable que la eficiencia del programa iba a ser mucho mayor con instrucciones diseñadas que con instrucciones generales, pues se realizan más operaciones en menos tiempo si se ejecutan procesos paralelos en el microcódigo que si se hace en ensamblador.